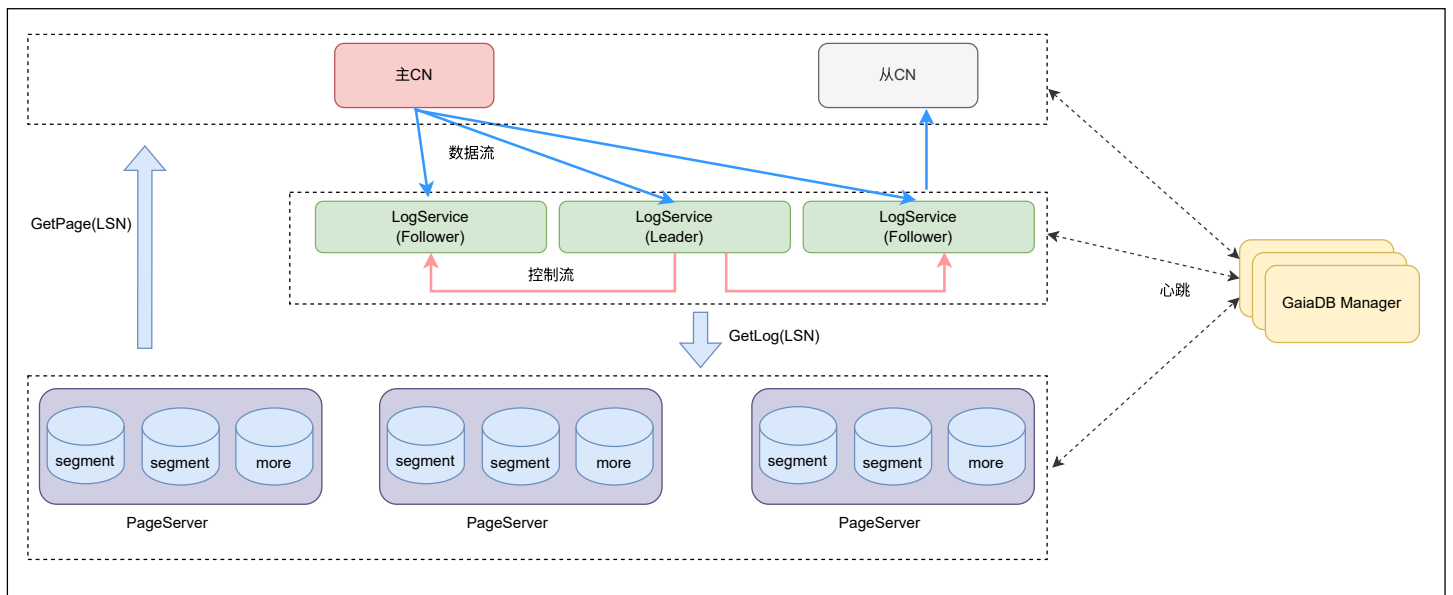


GaiaDB LogService层分享

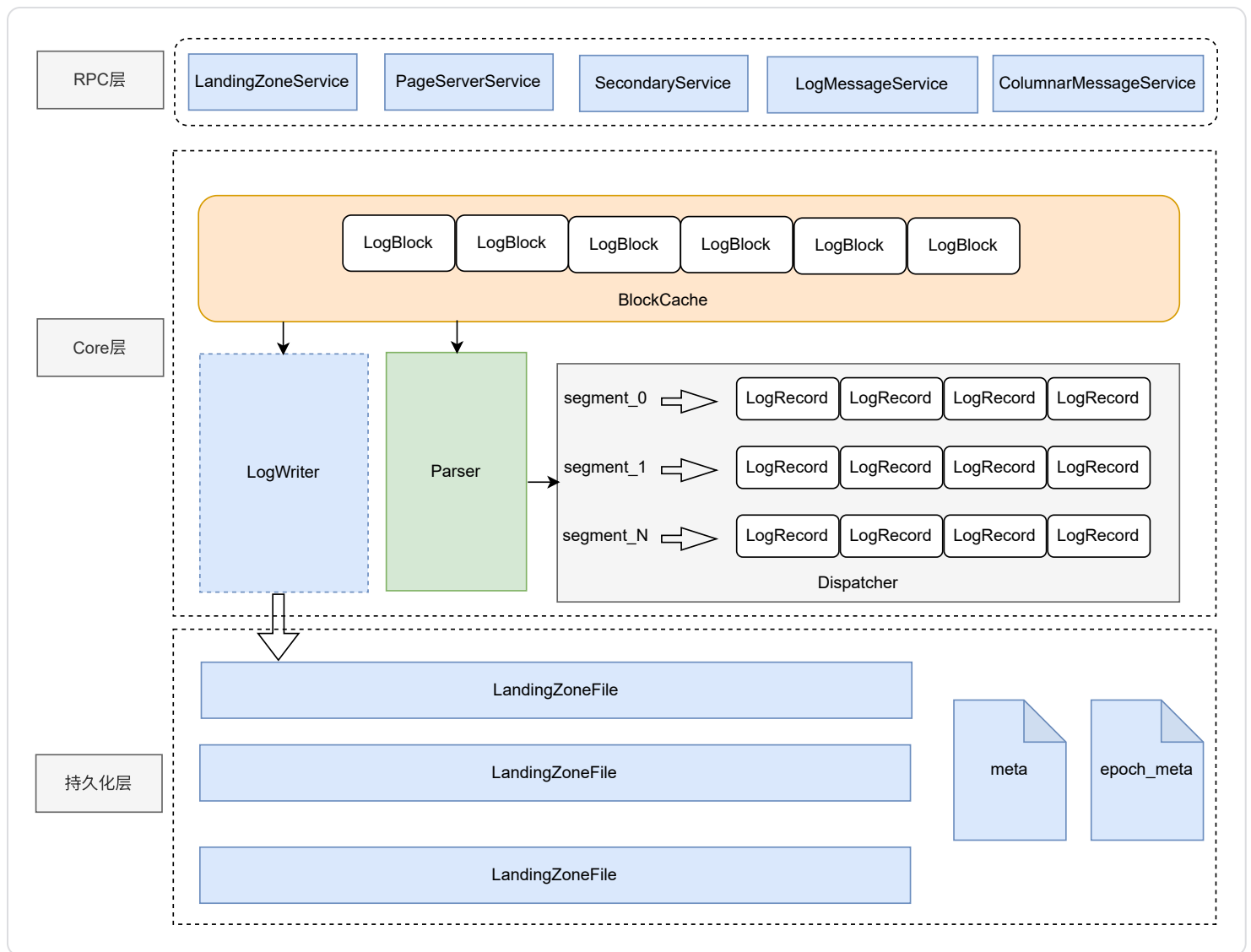
一、GaiaDB存储层架构

存储层主要涉及模块包括: LogService、PageServer与PageServerManager



- GaiaDB Manager: 实例中心节点，管理集群元数据，收集模块心跳及推进LSN，管理逻辑PageNo到物理Segment的映射。
- LogService: 主CN写的redo log的管理与持久化，供PageServer、从CN拉取
- PageServer: 集群存储节点，负责Segment的管理、物理Page的回放与存储、对CN提供RBIO

二、LogService模块



三、读写路径

3.1 写路径

主CN通过Disptach方式并发往3个LogService写Log，只要quorum 写成功(至少收到2个ACK)，即可认为这条redo log 写成功。LogService模块收到pwrite()的处理流程包括：

- 请求合法性校验: 包括请求中cluster_id、epoch/membership_epoch 参数，这里epoch有2个:
 - epoch 主要用于计算节点的成员变更，每次成员变更都会提升存储层epoch，防止脑裂。
 - membership_epoch主要用于LogService 成员变更，每次成员变更都会增加，这块复用了raft 复制组成员变更的逻辑，不再详细展开。
- 推进本地VDL(如需)
- 流控相关逻辑:
 - 最大并发写入请求数
 - BlockCache中缓存的Log空间是否已到阈值上限
- 将Log填充到BlockCache中
- 将Log LSN加入到LinkedBuf中，LinkedBuf是一个用于记录连续LSN LOG的数据结构，LogWriter会利用本结构计算Log间是否存在空洞(因quorum write导致缺失部分Log)，如需会基于类Gossip协议从LogService Peer 中读取缺失的Log

6. 等待LogWriter 将本Log写入LandingZone文件中，LogWriter写Log是顺序的，中间不允许空洞，换言之如果本LSN log写成功，那么之前全部Log都写成功
- 这样设计好处的，主CN在quorum write时候，不需要维护每条Log的写入状态，用于判定本Log是否quorum write成功，而直接利用不同LogService最后一个写success的Log LSN，通过简单计算得到VCL/VDL

i 几个注意点:

- LogWriter 写redo log到LandingZone文件，为提升IO效率与吞吐，会尝试一次写多条Log
- BlockCache 缓存Capacity是有上限的，当因CN或PageServer回放较慢，可能导致较老Log无法及时GC，导致BlockCache填满，进而影响写入服务

3.2 读路径

PageServer与从CN节点，都会从LogService拉取redo log，拉取请求都会提供一个起始LSN，LogService需返回从LSN开始的后续Log。二者区别在于Log范围，从CN需要拉取全部增量Log，而PageServer是以底层存储Segment为粒度，要求每次返回某个Segment的若干连续Log，因此LogService中内置了Parser/Dispatcher服务，会把全部Log根据所属Segment进行分类。

3.2.1 CN的GetLog

- 请求参数检查，cluster_id
- 根据start LSN从BlockCache依次读取LogBlock内容，RPC返回

3.2.2 PageServer的GetLog

- 参数合法性检查
- start LSN检查，要求start LSN $\geq$ gc LSN，因gc LSN 之前的Log可能已经不在BlockCache中
- 根据请求Segment ID从Dispatcher中获取本Segment的连续Log Record，并根据Record中描述LSN从BlockCache中读取Log，RPC返回。

四、GC与重启

4.1 GC

- 目的: BlockCache的缓存Capacity是有上限的，一旦写满就无法接受新写入请求，而GC目的在于及时推进gc_lsn，如Log end_lsn $\leq$ gc_lsn，则其对应缓存可以被覆盖。
- GC原则: BlockCache的Log主要服务于PageServer、从CN、Columnar等模块的GetLog请求，换言之如果某条Log这些模块都已不再需要，则不必缓存。因此PageServerManager会通过这些模块心跳收齐其最低LSN 水位线(比如PageServer 某条Log已经回放完成并持久化到Segment物理文件，则不再需要)，计算这些模块所需的最低LSN，这个最低LSN是逐步上涨的。LogService的GC线程会周期性从PageServerManager获取这个最低LSN，依次为依据做GC。
- GC执行: 某条Log可以被GC后，GC 线程并不会立马从BlockCache 中淘汰掉，而是采用Lazy evict的方式，当有新的写入时，且新写入的Log期望Slot被某条Log占用，而这条Log可以被GC，新Log会直接覆盖，达到类似GC目的

4.2 重启

LogService服务重启后，BlockCache缓存及Parser、Dispatcher 工作上下文会丢失，因此其会从PageServerManager拉到最低LSN，从这LSN开始读取LandingZone文件数据填充回BlockCache，Parser/Dispatcher也从本LSN恢复工作。

另一方面，LogService的持久化层会保存到raft snapshot中，如果发生LogService故障，需要迁移，这块直接复用了braft能力。